Building an App at Blockit's Level — Full Developer Guide

Everything in this document was measured from a shipping app (Blockit 2.2.7, com.hypenet.focused) — its resources, its decompiled view code, its sound engine, its motion constants. It is written as build instructions: follow the numbers and the patterns and you reproduce the feel, not just the look. Companion files live in this kit: fonts/ (the 4 typefaces), tokens/ (stock light + night colour files, full string table), views/widget/ (102 decompiled view sources), sound/ (the synth engine source), layouts/ (the app's own layout XMLs), SPEC.md (the one-page number sheet).

Stack facts to mirror: native Android, Kotlin/Java, no Jetpack Compose on the designed surfaces, zero native code, minSdk 28 / targetSdk 36. Every designed control is a custom View that draws itself on Canvas. Stock Material widgets appear only on undifferentiated screens (settings lists, dialogs).

---

1. Colour system — 19 tokens carry the whole app

Do not scatter hex values. Define exactly these slots in res/values/colors.xml and mirror them in res/values-night/colors.xml. Every view, layout and drawable in the app resolves colour through these names and nothing else (244 colour entries exist in the file; ~19 are app-owned — the rest are library defaults you leave alone).

Light (the reference set)

Token	Value	Job
colorBackground	#E0E0E0	Window/screen ground
colorSurface	#E0E0E0	Default surface (same as ground — screens are flat)
colorSurfaceSheet	#E0E0E0	Bottom sheets
colorSurfaceVariant	#DBDBDB	Recessed/inset areas, preset tiles
colorSurfaceBright	#FFFFFF	The one raised card (the big timer card, nav capsule)
colorPrimary	#252323	Ink. Buttons and primary controls are near-black, not coloured
colorOnPrimary	#FBF1EC	Content on ink
colorPrimaryContainer	#F55314	THE accent — hot orange, spent rarely (see §1.1)
colorOnPrimaryContainer	#471602	Content on the accent
colorSecondaryContainer	#615B5A	Selected pill in the bottom nav
colorOnSecondaryContainer	#F4EFEE	Content on the pill
colorOnSurface	#4E4948	Primary icon/glyph tone on surfaces
colorOnSurfaceVariant	#787170	Secondary text/glyphs
colorOutline	#615B5A	Strong hairlines, borders
colorOutlineVariant	#75615B5A	Hairlines (base colour at 0x75 alpha)
blockBarSurface	#2E2A26	The session/block bar — darkest element in the app
textColorPrimary	#2C2A29	Body text
textColorSecondary	#615B5A	Secondary text
nothing_red	#E30613-family red	Brand/dot-matrix accents only

Night

Token	Value
colorBackground / colorSurface	#201D1A
colorSurfaceSheet	#2E2A26
colorSurfaceVariant	#272320
colorSurfaceBright	#332A27
colorPrimary	#FFE4CF (flips to cream — ink inverts, doesn't grey)
colorOnPrimary	#1B110D
colorPrimaryContainer	#F47442 (accent warms up one step for dark ground)
colorSecondaryContainer	#FFB38F
colorOnSecondaryContainer	#40352E
colorOnSurface	#FBECE1
colorOnSurfaceVariant	#FAE4D3
colorOutline	#807A7A
colorOutlineVariant	#758F8888
textColorPrimary	#F8F5F5
textColorSecondary	#F1E8E6

1.1 The discipline that makes it work

• One accent, structurally. colorPrimaryContainer is the only saturated colour in the system. It is spent on: the session timeline bar, the sled/power active tiles, the armed state. If a screen shows the accent in more than ~3 places, the design is wrong.
• "Primary" is a neutral. Primary buttons are ink with cream content. Colour is never decoration; it always means state (active, armed, selected).
• Flat by default. Ground, surface and sheet are the same hex in light mode. Depth comes from exactly one raised tone (colorSurfaceBright, pure white) and one recessed tone (colorSurfaceVariant, −5 lightness). No shadows-as-design, no gradients on surfaces.
• Alpha is part of the token. Hairlines are the outline colour at 0x75 alpha, encoded in the token itself (#75...), so every hairline in the app is identical.
• Verify visually, slot by slot. When re-skinning, map from consumers: find which token each big surface actually reads (timer card → colorSurfaceBright, preset tiles → colorSurfaceVariant, nav pill → colorSecondaryContainer) before assigning values. A colour parked in a slot nothing paints does not exist.

---

2. Typography — three families, numbers are architecture

Fonts ship in the kit (fonts/): wire them as font resources (res/font/) and reference by token, never by file path, in views.

Family	Files	Used for
Google Sans	googlesansregular.ttf, googlesansbold.ttf	All words: labels, buttons, body, headings
Space Mono	spacemonoregular.ttf	Every number — timers, counts, stats, prices

| Nothing (dot-matrix) | nothing_regular.ttf | Logo/brand moments and big dot-matrix readouts only |

Rules:

• Numbers never render in the text font. If a string contains a quantity, it is Space Mono. Mixed labels keep words in Google Sans and the numeral in Space Mono (two typefaces inside one label is normal here).
• Hero numbers are sized like architecture, not text. The countdown digits are 148dp type on a 96dp column width (countdowncountertextsize / countdowncounterwidthsize dimens). When a number is the screen's subject, start at 100dp+ and lay the screen out around it.
• Dot-matrix is a spice. The Nothing font appears at brand moments and playful readouts; a full screen of it is unreadable and off-system.
• Letterforms do the hierarchy work: size + family + weight. There is no italic anywhere in the system.

---

3. Geometry — two radii, one gap, fixed heights

Measure	Value	Where
Corner radius, small	12dp	Tiles, chips, buttons, cells
Corner radius, large	24dp	Cards, sheets, the timer card
Content gap / padding unit	12dp	The universal spacer — margins are multiples of it
Bottom navigation height	80dp	Fixed; content respects it
FAB diameter	96dp	Oversized on purpose — it's a machine button
Bag/app cell	48dp cell, 28dp icon	Suitcase grid + session strip
Slider dot radius	14dp	Rotary/slider tracks
Dot-matrix tile	4dp	Tile-number rendering grid
Countdown digit size / column	148dp / 96dp	See §2

Screens are full-bleed stacked sections — edge-to-edge bands and grids, not cards floating on a background with margins. If you catch yourself adding a 16dp screen margin and floating cards, you have left the system.

---

4. The control pattern — how every Blockit view is built

This is the core of the whole document. There are ~97 custom views in it.mirko.widget (102 source files in the kit) and they all follow one anatomy. Learn the anatomy once; every control is an instance of it.

4.1 Anatomy (from RotaryWheel.java, 328 lines — the reference implementation)

```
public class RotaryWheel extends View {
  // 1. ALL drawing state is preallocated final fields. Nothing is created in onDraw.
  public final Paint paintDial   = new Paint(Paint.ANTI_ALIAS_FLAG);
  public final Paint paintTicks  = new Paint(Paint.ANTI_ALIAS_FLAG);
  public final Paint paintAccent = new Paint(Paint.ANTI_ALIAS_FLAG);
```

```
  public final Paint paintText   = new Paint(Paint.ANTI_ALIAS_FLAG);
  public final RectF  arcBounds  = new RectF();
  public final Path   path       = new Path();

  // 2. Services and animation are owned by the view.
  public final Vibrator vibrator;
  public ValueAnimator settleAnimator;
  public ValueAnimator fadeAnimator;

  // 3. Plain state fields — the view is its own model.
  float angle; float velocity; int value; boolean dragging;

  public RotaryWheel(Context ctx, AttributeSet attrs) {
    super(ctx, attrs);
    // resolve EVERY colour/font from resources here, once
    paintDial.setColor(ContextCompat.getColor(ctx, R.color.colorSurfaceVariant));
    // ...
    vibrator = (Vibrator) ctx.getSystemService(Context.VIBRATOR_SERVICE);
  }

  // 4. Feedback helper — one-shot, milliseconds (see §6)
  void tick(int ms) { vibrator.vibrate(VibrationEffect.createOneShot(ms, -1)); }

  @Override protected void onDraw(Canvas c) {
    // primitives only: drawArc / drawCircle / drawLine / drawRoundRect / drawText
    // zero allocation, zero resource lookups, zero string building
  }

  @Override public boolean onTouchEvent(MotionEvent e) {
    // the view interprets its own gestures end-to-end:
    // hit → drag with velocity → release → settle animator → detent ticks
    // calls invalidate() on itself only; never touches siblings
    return true;
  }
}
```

Non-negotiables visible in the real code:

• Paints are final fields created with new Paint(1) (ANTIALIASFLAG) in field initialisers/constructors — 4+ paints per view is normal (dial, ticks, accent, text). No new inside onDraw, ever.
• The view owns its ValueAnimators and its Vibrator. There is no central animation manager and no callback spaghetti into the Activity for feel.
• State change → invalidate() on itself only. A tick of the wheel never redraws the screen.
• Colours and fonts are resolved in the constructor from R.color/R.font tokens (§1, §2) — so a theme change is a resource change, not a code change.
• Touch is interpreted, not forwarded. Detents, fling velocity, snap/settle and the haptic tick all live inside the control. The Activity receives only the final semantic event (value chosen, switch armed).

• Sizes in the real files: hero controls run 250–550 lines (RombiKnob 541, BagTiles 407, SlideToStart 343, RotaryWheel 328, SledSwitch 302, TimelineBar 260). A control that matters is a few hundred lines. Budget for it.

4.2 The control set to build

Build these as self-contained Canvas views, in this order (each is in views/widget/ as a worked reference):

1. RotaryWheel — the time picker. A dial you drag rotationally; tick marks around the rim; detent haptic per step; release settles with curve B (§5). This is the hero control — the screen exists around it.
2. SlideToStart — the arming gesture. A thumb you drag the full width of its track to commit an action; partial drags spring back (OvershootInterpolator). Any consequential action in your app should require a physical-feeling commit like this, not a tap.
3. HapticPreset (74 lines — the smallest complete example) — a duration chip: rounded-rect (12dp), label + Space Mono numeral, press = scale/haptic, selected = accent or ink fill. Copy this class's shape for every simple control you invent.
4. BagTiles — a grid of 48dp cells (28dp icons) representing a set of chosen apps/items; empty cells render as outline slots; tap opens a picker (a stock z8/o0-style flow in the original). Cells are drawn, not inflated child views.
5. TimelineBar — a horizontal progress bar with segment ticks; the accent's main home. Rounded ends, hairline track in colorOutlineVariant.
6. SledSwitch / AdvanceLever — mechanical toggles: a sled that slides in a track with a lever throw, not a Material Switch. State changes are animated throws (260–420ms), with a clunk haptic (10ms).
7. RombiKnob — a diamond/rhombus knob for discrete levels; rotation snaps at detents.
8. TileNumberView / TileDisplay — numbers rendered as physical tiles on the 4dp dot grid (the "counter machine" readout). Use for streaks/scores.
9. TowerView pattern — the streak visual: bricks laid 7 per floor, floor = (days + 6) / 7; one brick per day, forever scrollable. Any long-term progress in your app deserves a physical accumulation metaphor like this, drawn the same way.
10. FallingSquaresView / DayFaders — celebration/ambient: a one-shot rain of 16 squares for milestone moments; faders for day cells. Ambient motion exists but is rare and earned.

Layouts then compose these views in XML (layouts/ in the kit) with the geometry of §3. No stock Material widget appears on a main screen.

---

5. Motion — three curves run the entire app

Define these once, in one constants class, and reference them everywhere (the original references its curve class 150+ times; nobody invents a per-animation curve):

```
public final class Curves {
  // A — the workhorse: hard snap out, long settle. Default for state changes.
  public static final Interpolator A =
    new PathInterpolator(0.2f, 0.0f, 0.0f, 1.0f);
  // B — rises fast, glides in. Wheel settle, value snapping.
```

```
  public static final Interpolator B =
      new PathInterpolator(0.05f, 0.7f, 0.1f, 1.0f);
  // C — slow swell, soft stop. Big surfaces, sheets, reveals.
  public static final Interpolator C =
      new PathInterpolator(0.3f, 0.0f, 0.8f, 0.15f);
  // Mechanical springs for levers/slides:
  //   OvershootInterpolator, AnticipateOvershootInterpolator (framework)
}
```

(In CSS/web terms these are cubic-bezier(0.2, 0, 0, 1), cubic-bezier(0.05, 0.7, 0.1, 1), cubic-bezier(0.3, 0, 0.8, 0.15) — the system ports to any platform verbatim.)

Duration family (ms) — nothing exceeds ~550

Duration	Use
180	Fades, colour/state cross-fades
260 (+160 delay where sequenced)	Reveals, chip/lever throws
280	Wheel settle after release
420	Card slides, medium transitions
520	The biggest transitions only (screen-level)

Rules:

• Every control animates itself with its own ValueAnimator (§4.1) and invalidates only itself per frame.
• Pick the duration from the family by distance travelled, not by taste: under a finger's width → 180–280; a card's height → 420; a screen → 520.
• If an animation needs a new curve, the design is wrong — rework the motion until A, B or C fits.

---

6. Haptics — clicks, not buzzes

The entire haptic vocabulary is VibrationEffect.createOneShot(ms, DEFAULT_AMPLITUDE) at four durations (29 one-shots + 21 waveforms exist in code; zero performHapticFeedback calls — every feel is hand-set per control):

Pulse	Feel	Fires on
1ms	Barely-there tick	Finest detents, slider micro-steps
4ms	Light click	Wheel detents, chip presses
8ms	Firm click	Detent landings, toggle throws
10ms	Clunk	Arming/commit moments (SlideToStart completion, lever lock)

Rules:

• If the user can hear the vibration motor, the pulse is too long. These are felt, not heard.
• One pulse per discrete event. Never vibrate continuously during a drag — tick at detent crossings only.
• Haptic and sound (§7) fire on the same event from the same call site inside the control, so the click

you feel and the click you hear are one gesture.

---

7. Sound — fully synthesised, no audio files

There is not one .mp3/.wav in the app. One engine class (sound/SoundSynth-h9-c.java in the kit — study it directly) generates PCM and writes it to a dedicated AudioTrack on its own thread, with AudioFocusRequest handled properly. The recipe:

```
partial weights (sine harmonics):  { 0.6, 0.28, 0.37, 2.2 }
decay envelope A (body):        multiply by exp(-0.0114) per sample step
decay envelope B (tail):        multiply by exp(-0.0313) per sample step
sample format:                  float PCM → AudioTrack, own writer thread,
                                volatile gain/state fields, SystemClock-timed triggers
```

How to use it:

• One synth, many voices. Retune the base frequency per control (wheel tick ≠ lever clunk ≠ arm confirmation) but keep the partial weights and both decay constants — that decay shape is the app's sonic signature: a short mechanical body with a fast, clean tail.
• Trigger from the control at the same moment as the haptic (§6). Volume is modest by design; these are machine clicks at conversation-with-hardware distance.
• Manage audio focus like a good citizen: request on first use, abandon on teardown; never fight the user's music.

---

8. Layout & resource organisation

• res/values/colors.xml + res/values-night/colors.xml — the §1 tokens, mirrored.
• res/values/dimens.xml — every number in §3 as a named dimen (countdowncountertextsize = 148dp style). Views read dimens; layouts reference them; no magic numbers in XML.
• res/drawable/ (258 XMLs in the original) — vector icons and shape/state drawables; they reference colour tokens only, so drawables are theme-free.
• res/anim/ (43 XMLs) — framework animations for the few non-custom transitions.
• res/layout/ (~45 app-owned of 131) — screens composed of §4 views in full-bleed section stacks; the bottom nav is an 80dp custom bar, not a Material BottomNavigationView.
• Strings: the full app runs on 682 strings. Write UI copy in the same voice — short, concrete, ritual-flavoured verbs for consequential actions (arm/start/commit over "OK"), plain nouns for objects, no exclamation marks, no emoji, no marketing adjectives inside the product.

---

9. Performance — beauty that doesn't jank is the whole game

• Zero allocation in onDraw — paints, paths, rects are fields (§4.1). Strings for readouts are pre-built on state change, not per frame.

• Invalidate narrowly — a control redraws itself; the screen never redraws wholesale for one control's animation.
• Ship baseline profiles (assets/dexopt/, generated with the Baseline Profile Gradle plugin for your dex) so first frames after install are pre-compiled. The original ships them; cold-start smoothness is part of the perceived quality.
• Primitives only — arcs, circles, lines, round-rects, text. No bitmap scaling in the draw path; icons are decoded once and cached by the loader, drawn as-is.
• A dropped frame is a design defect, not a performance ticket. Profile the hero screen first (wheel + countdown + strip animating together is the worst case).

---

10. Build order (do it in this sequence)

1. Tokens first. Colours (§1), dimens (§3), fonts (§2) as resources. No view code yet.
2. The constants class. Curves (§5), the haptic helper (§6), the sound engine (§7) — the feel layer, testable with three throwaway buttons.
3. HapticPreset (the 74-line control) — prove the §4 anatomy end-to-end on the smallest control.
4. The hero control (your RotaryWheel equivalent) + its screen skeleton: ground, hero number at 148dp, hero control, commit gesture (SlideToStart).
5. The set/grid control (BagTiles pattern), the progress control (TimelineBar), the mechanical toggles (SledSwitch, RombiKnob).
6. The accumulation visual (TowerView pattern) and one earned celebration (FallingSquaresView).
7. Night tokens, baseline profiles, then — only then — secondary screens, which may use stock widgets.

11. Acceptance checklist (measurable — check every line)

• [ ] Every colour in the app resolves to one of the 19 tokens; no hex literal exists in a view or layout.
• [ ] Exactly one saturated accent exists, and it appears only where it means state.
• [ ] Every number on every screen renders in the mono font; hero numbers ≥100dp.
• [ ] Only two corner radii exist app-wide: 12dp and 24dp.
• [ ] Every animation uses curve A, B or C and a duration from {180, 260, 280, 420, 520}.
• [ ] Every haptic is a 1/4/8/10ms one-shot; performHapticFeedback appears nowhere.
• [ ] No audio files in the APK; all UI sound comes from the synth with the §7 constants.
• [ ] onDraw in every custom view allocates nothing; paints are final fields.
• [ ] A consequential action requires a physical commit gesture, not a tap.
• [ ] Hero screen holds 60fps with all its controls animating simultaneously, on a mid-range device, cold start included.